CS-2002
Quiz 1

1 Question 1.(a): Answer

Any one of the following recurrences:

$$T(n) = 2T\left(\frac{3n}{4}\right) + O(n)$$

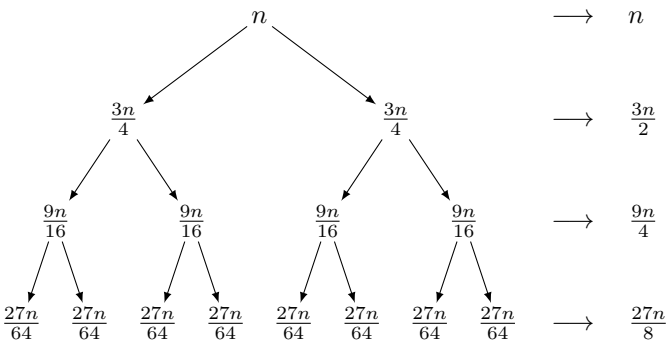
$$T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + O(n)$$

$$T(n) = T\left(\frac{n}{4} - a\right) + T\left(\frac{3n}{4} + a\right) + O(n)$$

2 Question 1.(b): Answer

2.1 Solving for $T(n) = 2T\left(\frac{3n}{4}\right) + O(n)$

2.1.1 Using Recursion Tree



Tree Height (h):

$$\left(\frac{4}{3}\right)^h = n \implies h \log_{4/3}\left(\frac{4}{3}\right) = \log_{4/3} n \implies h = O(\log_{4/3} n)$$

Work per Level:

- Level 0: n

- Level 1: $2 \cdot \left(\frac{3n}{4}\right) = \left(\frac{3}{2}\right) n$
- Level 2: $4 \cdot \left(\frac{9n}{16}\right) = \left(\frac{3}{2}\right)^2 n$
- Level i: $\left(\frac{3}{2}\right)^i n$

Total Cost: Summing over all levels:

$$\text{Total} = \sum_{i=0}^{h} n \left(\frac{3}{2}\right)^i = n \left[\frac{\left(\frac{3}{2}\right)^{h+1} - 1}{\frac{3}{2} - 1}\right] = O\left(n \left(\frac{3}{2}\right)^h\right) = O\left(n \left(\frac{3}{2}\right)^{\log_{4/3} n}\right)$$

Since $a^{\log_b c} = c^{\log_b a}$, we can write the following as well,

$$\left(\frac{3}{2}\right)^{\log_{4/3} n} = n^{\log_{4/3}(3/2)}$$

Thus:

$$\text{Total} = O\left(n \cdot n^{\log_{4/3}(3/2)}\right) = O\left(n^{1+\log_{4/3}(1.5)}\right)$$

So, it is bounded by approximately, $O\left(n^{2+\varepsilon}\right) \quad (\varepsilon \geq 0)$.

2.1.2 Using Substitution Method

Attempt 1: Assume $T(n) = O(n \log n) \leq cn \log n$
Substituting into the recurrence:

$$\begin{aligned} T(n) &\leq 2c \left(\frac{3n}{4}\right) \log \left(\frac{3n}{4}\right) + c'n \\ &= \frac{3}{2} cn \left(\log n + \log \frac{3}{4}\right) + c'n \\ &= \frac{3}{2} cn \log n + \frac{3}{2} cn \log \left(\frac{3}{4}\right) + c'n \end{aligned}$$

Since $\frac{3}{2} cn \log n > cn \log n$, the bound does not hold.
$\therefore$ **Assumption is incorrect.**

Attempt 2: Assume $T(n) = O(n^2) \leq c_1 n^2$
Substituting into the recurrence:

$$\begin{aligned} T(n) &\leq 2c_1 \left(\frac{3n}{4}\right)^2 + c_2 n \\ &= 2c_1 \left(\frac{9}{16}\right) n^2 + c_2 n \\ &= \frac{9}{8} c_1 n^2 + c_2 n \end{aligned}$$

Since $\frac{9}{8} c_1 n^2 > c_1 n^2$, the induction fails.
$\therefore$ **Assumption is incorrect.**

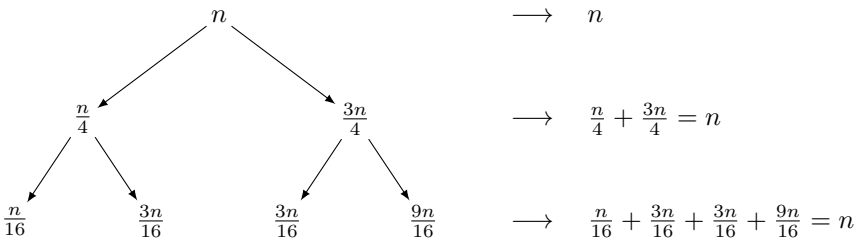
Attempt 3: Assume $T(n) = O(n^3) \leq cn^3$
Substituting into the recurrence:

$$\begin{aligned} T(n) &\leq 2c\left(\frac{3n}{4}\right)^3 + O(n) \\ &= 2c\left(\frac{27}{64}\right)n^3 + O(n) \\ &= \frac{27}{32}cn^3 + O(n) \end{aligned}$$

Since $\frac{27}{32} < 1$, for sufficiently large c and n, $\frac{27}{32}cn^3 + O(n) \leq cn^3$.
$\therefore$ **Assumption is correct.**

2.2 Solving for $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + O(n)$

2.2.1 Using Recursion Tree



Tree Height (h): The longest path to a leaf corresponds to the larger recursive call $T(3n/4)$:

$$\left(\frac{4}{3}\right)^h = n \implies h\log\left(\frac{4}{3}\right) = \log n \implies h = \frac{\log n}{\log(4/3)} = O(\log n)$$

Work per Level:

- Level 0: n
- Level 1: $\frac{n}{4} + \frac{3n}{4} = n$
- Level 2: $\frac{n}{16} + \frac{3n}{16} + \frac{3n}{16} + \frac{9n}{16} = n$
- Cost at each full level is $O(n)$.

Total Cost:

$$\text{Total Work} = O(n) \times O(\log n) = O(n \log n)$$

2.2.2 Using Substitution Method

Inductive Hypothesis: Assume $T(n) = O(n \log n) \leq c_1 n \log n$.

$$\begin{aligned} T(n) &= c_1\left(\frac{n}{4}\right)\log\left(\frac{n}{4}\right) + c_1\left(\frac{3n}{4}\right)\log\left(\frac{3n}{4}\right) + c_2 n \\ &= c_1\left(\frac{n}{4}\right)[\log n - \log 4] + c_1\left(\frac{3n}{4}\right)\left[\log n - \log\frac{4}{3}\right] + c_2 n \end{aligned}$$

Collecting the dominant terms:

$$\begin{aligned}\text{Dominant Term} &= c_1 n \log n \left(\frac{1}{4}\right) + c_1 n \log n \left(\frac{3}{4}\right) \\ &= c_1 n \log n \left(\frac{1}{4} + \frac{3}{4}\right) \\ &= c_1 n \log n = O(n \log n)\end{aligned}$$

The negative constant terms easily offset the $+c_2 n$ work for a sufficiently large choice of c_1.
$\therefore$ **The guess is verified:** $T(n) = O(n \log n)$.

2.3 Solving for $T(n) = T\left(\frac{n}{4} + a\right) + T\left(\frac{3n}{4} - a\right) + O(n)$

2.3.1 Using Substitution Method

Inductive Hypothesis: Assume $T(n) = O(n \log n) \leq c_1 n \log n$.

Since the smaller part can be as big as $n/2$ (otherwise it won't be the smaller part)

$$\log\left(\frac{n}{4} + a\right) \leq \log\left(\frac{n}{2}\right) = \log n - \log 2$$

The larger part can be $3n/4$ at most but it is a less than $3n/4$

$$\log\left(\frac{3n}{4} - a\right) \leq \log\left(\frac{3n}{4}\right) = \log n - \log\left(\frac{4}{3}\right)$$

Substituting into the recurrence:

$$T(n) \leq c_1\left(\frac{n}{4} - a\right)(\log n - \log 2) + c_1\left(\frac{3n}{4} + a\right)\left(\log n - \log\frac{4}{3}\right) + c_2 n$$

$$\begin{aligned}\text{Dominant Term} &= c_1\left(\frac{n}{4} + a\right)\log n + c_1\left(\frac{3n}{4} - a\right)\log n \\ &= c_1\left(\frac{n}{4} + a + \frac{3n}{4} - a\right)\log n \\ &= c_1 n \log n = O(n \log n)\end{aligned}$$

$\therefore$ **Guess successful:** $T(n) = O(n \log n)$.

Question 2

Each of divide, combine and conquer carries 0.5 marks.

(a) Divide. If we divide the two subproblems to have roughly $n/2$ points by splitting the points with $x = x_d$, then, a point with $x < x_d$ may end up in the right subproblem.

Conquer. The recursive calls can still be made on the two subproblems. However, if the subproblems are obtained merely by their positions in Arr, they do not necessarily correspond to the geometrically left and right parts of the plane. Hence, the geometric property required by the standard closest-pair algorithm is lost.

Combine. The standard combine step assumes

$$p \in L \Rightarrow x(p) \leq x_d, \qquad p \in R \Rightarrow x(p) > x_d.$$

Therefore, any closest pair having one point in each subproblem must lie in the vertical strip around $x = x_d$. However, if a point with $x < x_d$ can belong to the right subproblem, this property no longer holds. A pair may lie entirely on the same geometric side of x_d, so the usual combine argument is no longer valid.

(b) The issues above can be resolved by spending $O(n)$ time at the divide step.

Divide. Scan Arr once and partition the points into

$$L' = \{p : x(p) < x_d\}, \qquad M = \{p : x(p) = x_d\}, \qquad R' = \{p : x(p) > x_d\}.$$

Since more than $n/2$ points belong to M, we can distribute an appropriate number of the points in M between the two subproblems so that $|L| = \left\lfloor \frac{n}{2} \right\rfloor$, $\quad |R| = \left\lceil \frac{n}{2} \right\rceil$. This partition requires only $O(n)$ time.

Importantly, all points with $x < x_d$ are kept in the left subproblem and all points with $x > x_d$ are kept in the right subproblem. Only points with $x = x_d$ are split between the two sides.

Conquer. Recursively find the closest pair in each subproblem. Let the resulting distances be δ_L and δ_R, and set

$$\delta = \min\{\delta_L, \delta_R\}.$$

Since the two subproblems have sizes roughly $n/2$, the recursion remains balanced.

Combine. We must also consider a closest pair with one point in each subproblem. Consider the vertical strip

$$S = \{p : |x(p) - x_d| < \delta\}.$$

All points with $x = x_d$ are included in this strip, regardless of which subproblem they were assigned to. Hence, closest cross-subproblem pairs are not missed.

We then use the standard closest-pair combine procedure on the points in this strip, considering the points in increasing order of their y-coordinates. The combine step takes $O(n)$ time.

Therefore, the recurrence is

$$T(n) = 2T(n/2) + O(n),$$

and hence

$$\boxed{T(n) = O(n \log n)}.$$

Model Solution: Inorder Traversal of a Binary Tree

[3 Marks]

Each node x has fields $x.left$, $x.right$ and $x.key$, and a missing child is written NIL. An empty tree is NIL and has 0 nodes.

(a) Recursive algorithm [1 mark]

```
INORDER(x):
  if x ≠ NIL:
    INORDER(x.left)
    print x.key
    INORDER(x.right)
```

The whole tree is traversed by the call INORDER(*root*). The test against NIL is the base case: an empty subtree prints nothing and the call returns.

(b) Recurrence relation [1 mark]

Let $T(n)$ be the running time of INORDER(x) when the subtree rooted at x has n nodes, and let k be the number of nodes in the left subtree of x. Then $0 \le k \le n-1$, and the value of k is decided by the shape of the tree, not by n. For $n \ge 1$,

$$T(n) = T(k) + T(n-k-1) + c, \qquad T(0) = d.$$

Justification. Apart from the two calls, the procedure performs one test against NIL, one print and two call set-ups, a fixed number of steps independent of n, giving the constant c; an empty call performs only the failed test, giving the constant d. The n nodes consist of the root together with the two subtrees, so the right subtree has $n-k-1$ nodes, and the two calls cost $T(k)$ and $T(n-k-1)$.

(c) Solving the recurrence [1 mark]

Claim. $T(n) = (c+d)\,n + d$, hence $T(n) = O(n)$.

Proof by induction on n. For $n = 0$ the claim reads $T(0) = d$, which holds. Let $n \ge 1$ and assume the claim for all smaller values. Both k and $n-k-1$ are at most $n-1$, so the assumption applies to both:

$$\begin{aligned} T(n) &= T(k) + T(n-k-1) + c \\ &= \big[(c+d)k + d\big] + \big[(c+d)(n-k-1) + d\big] + c \\ &= (c+d)(n-1) + 2d + c \;=\; (c+d)n + d. \end{aligned}$$

The terms containing k cancel, so the same expression is obtained for every k and no assumption on the shape of the tree is used. As c and d are constants, $T(n)$ grows linearly and $T(n) = O(n)$.

Question 4

Student A was solving a Dynamic Programming problem involving two strings

$$X = x_1 x_2 \cdots x_m$$

and

$$Y = y_1 y_2 \cdots y_n.$$

Unfortunately, Student B lost the question paper and was able to recover only the following recurrence:

$$DP[i][j] = \begin{cases} 0, & \text{if } i = 0 \text{ or } j = 0, \\ DP[i-1][j-1] + 1, & \text{if } x_i = y_j, \\ \max\{DP[i-1][j], DP[i][j-1]\}, & \text{if } x_i \neq y_j. \end{cases}$$

(i) What problem is Student A most likely solving?

1.5 Marks

Answer:

Student A is solving the **Longest Common Subsequence (LCS)** problem.

The recurrence is exactly the standard Dynamic Programming recurrence for finding the length of the longest subsequence that occurs in both strings X and Y.

(ii) What does $DP[i][j]$ represent?

1.5 Marks

Answer:

$DP[i][j]$ represents the **length of the Longest Common Subsequence** between the prefixes

$$X[1 \ldots i] = x_1 x_2 \cdots x_i$$

and

$$Y[1 \ldots j] = y_1 y_2 \cdots y_j.$$

In other words,

$$\boxed{DP[i][j] = \text{length of the LCS of } X[1 \ldots i] \text{ and } Y[1 \ldots j].}$$

The recurrence can be understood as follows:

- If $i = 0$ or $j = 0$, one of the strings is empty, so the LCS length is 0.
- If $x_i = y_j$, the two characters can be part of the LCS. Hence,
$$DP[i][j] = DP[i-1][j-1] + 1.$$
- If $x_i \neq y_j$, the last character of at least one of the two prefixes cannot be part of the same LCS. Therefore, we consider both possibilities:
$$DP[i][j] = \max\{DP[i-1][j], DP[i][j-1]\}.$$

Q5 : Solution

Algorithm Description (2 Marks)

Let $G = (P, E)$ be the graph where vertices are the points $P = \{p_1, p_2, \ldots, p_n\}$ and edges E represent adjacency. We are given k colors. To apply the divide-and-conquer paradigm (specifically, a decrease-and-conquer variation), we maintain a set of candidate colors $\mathcal{C}(v) \subseteq \{1, 2, \ldots, k\}$ for each vertex $v \in P$. Initially, $\mathcal{C}(v) = \{1, \ldots, k\}$ for all v.

- **Base Case:** If $P = \emptyset$, return `True` (an empty graph is trivially colorable).
- **Divide Step:** Select an arbitrary single point $p \in P$. Partition the set of points into $\{p\}$ and the remaining points $P' = P \setminus \{p\}$.
- **Conquer Step:** Iterate through each available color $c \in \mathcal{C}(p)$. For each color c:
  1. Assign color c to p.
  2. **Propagate Constraints:** For every point $v \in P'$ such that $(p, v) \in E$, remove c from $\mathcal{C}(v)$.
  3. Recursively call the algorithm on P' with the updated candidate colors. If the recursive call returns `True`, return `True` immediately.
  4. **Backtrack:** If the recursive call returns `False`, restore c to the candidate colors $\mathcal{C}(v)$ of p's neighbors.
- **Combine Step:** The combine step is an implicit logical `OR` operation across the recursive branches. If all candidate colors $c \in \mathcal{C}(p)$ lead to a `False` evaluation from the recursive calls (or if $\mathcal{C}(p) = \emptyset$), return `False`.

Recurrence Relation (1 Mark)

Let $n = |P|$ be the number of points.

- **Divide:** Selecting a point and updating/backtracking the candidate colors for p's neighbors takes $O(n)$ time in the worst case.
- **Conquer:** In the worst case, the algorithm makes up to k recursive calls on subproblems of size $n - 1$. Total time: $kT(n-1)$.
- **Combine:** Returning the logical `OR` of the boolean results takes $O(1)$ time.

This yields the following recurrence for the worst-case running time $T(n)$, ordered sequentially by the steps:

$$T(n) = \underbrace{O(n)}_{\text{Divide}} + \underbrace{kT(n-1)}_{\text{Conquer}} + \underbrace{O(1)}_{\text{Combine}}$$

Combining the asymptotic terms gives:

$$T(n) = kT(n-1) + O(n)$$

with the base case $T(1) = O(1)$.

Running Time Analysis (1 Mark)

To solve the recurrence, we expand it iteratively:

$$\begin{aligned} T(n) &= k\big(kT(n-2) + O(n-1)\big) + O(n) \\ &= k^2 T(n-2) + k \cdot O(n-1) + O(n) \\ &= k^i T(n-i) + \sum_{j=0}^{i-1} k^j \cdot O(n-j) \end{aligned}$$

Continuing this expansion until the base case $i = n-1$:

$$T(n) = k^{n-1} T(1) + \sum_{j=0}^{n-2} k^j \cdot O(n-j)$$

Since $k \geq 2$, the summation is a geometric series that is strictly dominated by its largest terms (where j is close to n). The geometric growth bounds the sum to $O(k^n)$. Along with the base case term $k^{n-1} \cdot O(1) = O(k^{n-1})$, the overall asymptotic worst-case running time of the algorithm is:

$$T(n) = O(k^n)$$